Instructivo de Práctica — Sesión 9: Asistente ExploraIA

📌 **Qué vas a entregar:** al terminar esta práctica, ExploraEC tiene una pestaña «Asistente» donde preguntas por los gastos de tu viaje («¿en qué categoría gasto más?», «¿cuánto me queda en comida?») y puedes pedir que registre un gasto. Está conectada a un **proxy** propio (`mock-server/`) que llama a **Gemini** con una clave gratuita que tú creas hoy; la app no lleva ninguna clave de IA, envía solo el **resumen por categoría** y pide **confirmación** antes de crear nada. Además, el proxy se conecta al **servidor MCP del backend** (el del módulo 2) para pedir tu resumen de gastos, y lo compruebas con una **prueba de extremo a extremo**. Es la última funcionalidad nueva del curso: la Sesión 10 pule y empaqueta lo construido.

Objetivo

Crear una clave gratuita de IA, guardarla solo en un proxy intermedio y conectar la pantalla del Asistente a ese proxy y al backend de gastos de las Sesiones 6-8, con los mismos patrones asíncronos y de manejo de errores ya vistos, aplicando los principios de la teoría: la clave de IA vive solo en el servidor, datos mínimos y una persona que confirma las acciones. Al final, hacer que el proxy use el servidor MCP del backend (una demo de lo que la teoría explica como agente y MCP) y comprobar toda la cadena con una prueba de extremo a extremo.

Requisitos previos

- Proyecto `exploraec` de la Sesión 8 (inicio de sesión, token seguro y CRUD de gastos), guardado en git.
- Una cuenta personal de Google (para crear la clave gratuita en el Paso 0).
- Docker y el repositorio del backend `proyecto-curso-spec-kit` (el Paso 1 trae los comandos para encenderlo).
- Node.js 18 o superior, solo para el proxy (el Paso 1 comprueba la versión).
- Un emulador Android creado (la práctica usa los valores por defecto del emulador: no hay que configurar direcciones).

Cómo trabajar esta práctica

Igual que en la Sesión 8: todo el código ya está en la rama; **tú solo descomentas** los bloques `TODO(sesion-09) Paso N`. Selecciona las líneas comentadas que están **debajo** del `TODO` (no el `TODO` ni el «Por qué»), presiona `Ctrl + /` (`Cmd + /` en Mac) y, después, presiona `R` (mayúscula) en la terminal de `flutter run`.

Usarás cuatro terminales:

Terminal	Carpeta	Para qué
A — backend	`exploraec-curso/proyecto-curso-spec-kit`	`docker compose up` (queda abierta)
B — app	`exploraec-curso/exploraec`	`flutter run` (queda abierta)
C — comandos	`exploraec-curso/exploraec`	`git`, `flutter test`, `curl`
D — proxy	`exploraec-curso/exploraec` (luego entra a `mock-server`, Paso 2)	`node mock-server.js`, con tu clave en una variable de entorno (queda abierta)

Si se agota la cuota gratuita

El plan gratuito de Google permite pocas consultas al día. Si el Asistente (o el `curl` del Paso 3) muestra *«Se alcanzó el límite gratuito de consultas a la IA…»*, no pierdes la práctica: todas las preguntas de esta guía funcionan también en el **modo de reglas** del proxy, que responde sin internet y con el mismo contrato.

Para pasar al modo de reglas, en la terminal D:

1. Detén el proxy con `Ctrl + C`.
2. Borra la clave de esa terminal (en Linux y Mac):

```
unset GEMINI_API_KEY
```

En Windows (PowerShell):

```
Remove-Item Env:GEMINI_API_KEY
```

3. Vuelve a encender el proxy:

```
node mock-server.js
```

El mensaje debe decir `modo REGLAS`. En ese modo el asistente solo reconoce unas pocas frases (están en `mock-server/README.md`); las de esta guía están entre ellas.

Tiempo estimado

Total: 60 minutos

Paso	Descripción	Tiempo
0	Crear tu clave gratuita de Google	7 min
1	Encender todo desde cero y recap de la Sesión 8	7 min
2	Traer la rama de hoy y encender el proxy con tu clave	7 min
3	Conocer el contrato del proxy sin la app	5 min
4	Conectar el Asistente con el proxy	8 min
5	Confirmación humana antes de registrar	8 min
6	Errores: límite de 500 y proxy apagado	7 min
7	Datos mínimos: qué viaja al proxy	4 min
8	El proxy consulta el MCP del backend y prueba de extremo a extremo	7 min
9 *(opcional)*	Preguntas sugeridas — no cuenta dentro de los 60 min	8 min
Cierre	Guardar tu práctica: commit y push — no cuenta dentro de los 60 min	3 min

Paso 0 — Crear tu clave gratuita de Google (7 min)

Por qué se hace

Un asistente de IA real necesita la **clave** de un proveedor. Hoy usarás la de Google (Gemini), que se crea gratis y sirve para desarrollo. Esa clave es un secreto: **nunca irá en la app**. Vivirá solo en el proxy, que es lo que explica la teoría de hoy.

El plan gratuito permite **pocas consultas al día**. Por eso la práctica está pensada para gastarlas con cuidado y trae un plan B (sección «Si se agota la cuota gratuita») si se agotan.

Instrucciones

0.1 — Abrir AI Studio. En el navegador entra a esta dirección:

```
https://aistudio.google.com/apikey
```

0.2 — Iniciar sesión. Entra con tu cuenta de Google. Usa una cuenta **personal**; si tu cuenta institucional no te deja crear la clave, usa una personal. Se exige ser mayor de 18 años.

0.3 — Aceptar los términos. Si la página los muestra, acéptalos.

0.4 — Crear la clave. Pulsa **Create API key**. Si te pide elegir un proyecto, crea uno nuevo o usa el que te sugiera.

0.5 — Copiar la clave. Cópiala. Si la pierdes, vuelve a esta página y cópiala de nuevo.

0.6 — No guardarla en ningún archivo. No la pegues en el código, en un `.env` del proyecto ni en un mensaje. En el Paso 2 la pegarás solo en una terminal.

Resultado esperado

Tienes una clave copiada y sabes que no debe escribirse en ningún archivo del proyecto. Sabes que el plan gratuito tiene pocas consultas al día, y que Google puede usar el contenido de esas consultas para mejorar sus productos.

Errores comunes

- **No aparece el botón *Create API key* o muestra un mensaje de permisos**. *Solución:* tu cuenta (casi siempre una institucional) no tiene permiso; usa una cuenta personal de Google.
- **Dice que no cumples la edad mínima**. *Solución:* la cuenta debe pertenecer a una persona mayor de 18 años; usa otra cuenta.
- **No puedes crear la clave por ningún motivo**. *Solución:* sigue la práctica sin clave: el

proxy tiene un modo de reglas (ver «Si se agota la cuota gratuita») que usa el mismo contrato. Avisa a tu docente.

✅ Antes de continuar, confirma que...

- ☐ Tienes una clave copiada (o sabes que irás con el modo de reglas).
- ☐ No la escribiste en ningún archivo.

Paso 1 — Encender todo desde cero y recap de la Sesión 8 (7 min)

Por qué se hace

El asistente se apoya en tu sesión (el token) y en tus gastos: necesita backend, emulador y app encendidos, y al menos 3 gastos en 2 categorías para tener algo que analizar. Este paso se hace con tu proyecto **tal como quedó en la Sesión 8**, antes de traer la rama de hoy.

Instrucciones

1.1 — Guardar la Sesión 8. En la terminal C:

```
git status
```

Si aparecen cambios sin guardar, guárdalos (el Punto de partida sobrescribe `lib/`):

```
git add -A
git commit -m "Sesión 8"
```

1.2 — Levantar el backend. En la terminal A, dentro de `proyecto-curso-spec-kit`, y déjala abierta:

```
docker compose up
```

Termina cuando veas `Uvicorn running on http://0.0.0.0:8000`. (Si no tienes la carpeta del backend, usa los comandos del Paso 0.2 de la Sesión 8.)

1.3 — Comprobar el backend. En la terminal C:

```
curl http://localhost:8000/health
```

Debe responder `{"status":"ok","db":"ok"}`.

1.4 — Encender el emulador. En la terminal C (`<id>` es la primera columna de la lista):

```
flutter emulators
flutter emulators --launch <id>
```

1.5 — Ejecutar tu app. En la terminal B, con el emulador encendido (y déjala abierta):

```
flutter run
```

1.6 — Entrar y revisar tus gastos. En la app: pestaña **Gastos**, inicia sesión y comprueba que tienes al menos 3 gastos en 2 categorías. Si te faltan, créalos con el botón **+**.

1.7 — Comprobar Node.js. En la terminal C:

```
node --version
```

Debe mostrar la versión 18 o superior.

1.8 — Responder en una frase: ¿por qué la clave que acabas de crear no puede ir dentro de la app?

Resultado esperado

Backend, emulador y app encendidos; ves tus gastos (3 o más, en 2 categorías); `node --version` responde 18 o más; y puedes explicar que una clave dentro de una app compilada termina en manos de cualquiera, por eso hoy se usa un proxy.

Errores comunes

- **`Cannot connect to the Docker daemon`**. *Solución:* abrir Docker Desktop y esperar a que diga *running*; en Linux, `sudo systemctl start docker`.
- **`node: command not found`**. *Solución:* instalar Node.js (versión LTS) desde nodejs.org y abrir una terminal nueva.
- **La pestaña Gastos pide iniciar sesión con «Tu sesión caducó»**. *Solución:* el token dura 30 minutos; inicia sesión de nuevo.

✅ Antes de continuar, confirma que...

- ☐ `curl http://localhost:8000/health` responde `ok`.
- ☐ Tu app corre y ves al menos 3 gastos en 2 categorías.
- ☐ `node --version` responde 18 o superior.

Punto de partida

Dentro de tu proyecto `exploraec`, en la terminal C:

```
git fetch starter
git checkout starter/sesion-09 -- lib test pubspec.yaml PLACEHOLDERS.md mock-server
flutter pub get
```

La rama trae el servicio del asistente, la pantalla con sus bloques por descomentar, el proxy (`mock-server/`) y 7 pruebas nuevas (junto con las 15 de la Sesión 8 suman 22). El detalle de cada bloque está en `PLACEHOLDERS.md`.

Paso 2 — Traer la rama de hoy y encender el proxy con tu clave (7 min)

Por qué se hace

La app no habla con ningún modelo de IA: habla con un **proxy**, que es quien guarda la clave y llama al modelo. Tu clave solo existirá en la terminal del proxy, como una **variable de entorno**: no se escribe en ningún archivo y se borra al cerrar la terminal.

> **Qué es el proxy y por qué lo usamos hoy**
>
> - **Qué es:** un servidor propio y pequeño que se pone en el medio: la app le pregunta a él, y él le pregunta al modelo de IA.
> - **Por qué existe:** la clave del modelo es un secreto. Si estuviera dentro de la app, cualquiera que la descargue podría sacarla y usarla a tu nombre. En el proxy, la clave no llega a ningún teléfono.

> - **Qué más hace:** decide qué datos salen hacia el modelo (solo el resumen por categoría), revisa lo que responde la IA antes de dárselo a la app y es el lugar donde se pueden poner límites de uso.
> - **Si la clave se filtra:** se cambia en un solo lugar, sin publicar una nueva versión de la app.
> - **Hoy es una demostración:** corre en tu computador. En un proyecto real estaría en un servidor de la organización.
>
> La teoría de hoy lo explica con diagramas (diapositivas «Arquitectura de hoy» y «La solución: un servidor intermedio»).

Instrucciones

2.1 — Traer la rama. Ejecutar el **Punto de partida** de arriba.

2.2 — Comprobar las pruebas. En la terminal C:

```
flutter test
```

Deben dar **19 pasando y 3 fallando** (las 3 son las del Asistente: todavía no descomentas nada).

2.3 — Entrar a la carpeta del proxy. En la terminal D, y déjala abierta:

```
cd mock-server
```

2.4 — Definir la clave. En la misma terminal D, cambia `pega-aquí-tu-clave` por tu clave. En Linux y Mac:

```
export GEMINI_API_KEY="pega-aquí-tu-clave"
```

En Windows (PowerShell):

```
$env:GEMINI_API_KEY="pega-aquí-tu-clave"
```

2.5 — Encender el proxy. En la terminal D:

```
node mock-server.js
```

2.6 — Comprobar el mensaje. La terminal D debe mostrar:

```
Proxy del asistente (modo GEMINI, modelo gemini-3.5-flash-lite, clave leída del
entorno) escuchando en http://localhost:3000
Desde el emulador Android, la app debe apuntar a http://10.0.2.2:3000
```

2.7 — Reiniciar la app. En la terminal B presiona `R`. Aparece una pestaña nueva: **Asistente**.

Resultado esperado

`flutter test` da 19 pasando y 3 fallando; el proxy dice `modo GEMINI` y no imprime tu clave; la app tiene la pestaña Asistente (con el campo de texto y el botón «Preguntar», que todavía no hace nada).

Errores comunes

- **El mensaje dice `modo REGLAS`**. *Solución:* la variable no estaba definida en esa terminal. Detén el proxy con `Ctrl + C`, repite el 2.4 y el 2.5 en la **misma** terminal.
- **`Error: listen EADDRINUSE: address already in use :::3000`**. *Solución:* otro programa usa el puerto 3000; ciérralo con `Ctrl + C` y vuelve a encender el proxy.
- **`flutter test` falla por otra razón (error de compilación)**. *Solución:* confirmar que `flutter pub get` se ejecutó después del Punto de partida.

✅ Antes de continuar, confirma que...

- ☐ `flutter test` muestra 19 pasando y 3 fallando.
- ☐ La terminal D muestra `modo GEMINI` (o `modo REGLAS` si no pudiste crear la clave).
- ☐ Ves la pestaña **Asistente** en la app.

Paso 3 — Conocer el contrato del proxy sin la app (5 min)

Por qué se hace

Antes de conectar nada conviene ver el contrato con tus propios ojos: qué entra y qué sale del proxy. Es la misma disciplina que ya usaste con el backend de gastos, aplicada ahora a un

contrato propio. Esta prueba gasta **una** consulta del plan gratuito.

Instrucciones

3.1 — Pedir que registre un gasto. En la terminal C, desde la carpeta de tu proyecto (en PowerShell usa `curl.exe` en vez de `curl`):

```
curl -X POST -H "Content-Type: application/json" -d @mock-server/ejemplo-registro.json
 http://localhost:3000/asistente
```

3.2 — Leer la respuesta. Llega un JSON con dos campos: `respuesta` (el texto de la IA) y `accion` (la descripción, el monto y la categoría que propone registrar). El texto exacto puede variar: la IA no responde siempre igual.

3.3 — Entender que solo propone. El proxy **no** crea ningún gasto: solo propone. Quien decide es la persona, y quien valida es el backend.

3.4 — Mirar la terminal D. Muestra el tamaño y los **nombres** de los campos recibidos (`consulta, resumen`), nunca su contenido ni tu clave.

3.5 — Responder en una frase: ¿qué datos tuyos NO aparecen en lo que recibió el proxy?

Resultado esperado

La respuesta trae `respuesta` y `accion`, y la terminal D muestra una línea con `campos: consulta, resumen`. Puedes decir que no viajan las descripciones de los gastos, las fechas ni tu correo: solo la pregunta y los totales por categoría.

Errores comunes

- **`curl: (7) Failed to connect`**. *Solución:* el proxy no está encendido (Paso 2).
- **`Could not open file ... for reading`** / **`no such file`**. *Solución:* ejecuta el comando desde la carpeta raíz de tu proyecto, no desde `mock-server`.
- **En PowerShell sale un error raro con `-H` o `-d`**. *Solución:* usa `curl.exe`; `curl` a secas es otro comando en PowerShell.
- **`{"detail":"El servicio de IA rechazó la clave…"}`**. *Solución:* la clave está mal copiada. Repite el 2.4 con la clave correcta y vuelve a encender el proxy.
- **`{"detail":"Se alcanzó el límite gratuito…"}`**. *Solución:* sigue con el plan B de la sección «Si se agota la cuota gratuita»: el modo de reglas.

✅ Antes de continuar, confirma que...

- ☐ Recibiste una respuesta con `respuesta` y `accion`.

- ☐ La terminal D registró la petición sin mostrar su contenido ni tu clave.
- ☐ Puedes decir qué datos no viajan al proxy.

Paso 4 — Conectar el Asistente con el proxy (8 min)

Por qué se hace

La pantalla ya tiene el campo de texto, el botón, el estado de carga y el aviso «Sugerencia de IA». Falta lo esencial: al preguntar, pedir el **resumen por categoría** al backend (con tu token) y enviarlo al proxy junto con la pregunta, en vez de mandar toda la lista de gastos.

Instrucciones

4.1 — Descomentar. En `lib/screens/assistant_screen.dart`, descomentar el bloque `TODO(sesion-09) Paso 4` (3 líneas, dentro de `_preguntar`).

4.2 — Aplicar. En la terminal B, presionar `R`.

4.3 — Preguntar. En la pestaña **Asistente** escribe `¿En qué categoría gasto más?` y toca **Preguntar**. Aparece la respuesta (esta pregunta gasta una consulta gratuita).

4.4 — Comprobar los datos. Abre la pestaña **Gastos** y confirma que la categoría y el total de la respuesta coinciden con los tuyos. Las palabras pueden variar de un intento a otro; los datos, no.

4.5 — Probar otra pregunta (opcional). Si te quedan consultas, escribe `¿Cuánto me queda en comida?`: el valor es el límite de 500 menos lo que gastaste en comida.

4.6 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

La respuesta coincide con tus datos reales. La terminal D muestra una petición por cada pregunta. `flutter test` da **20 pasando y 2 fallando**.

Errores comunes

- **La respuesta no aparece y no hay error**. *Solución:* falta descomentar el bloque, o no presionaste `R` (mayúscula).

- **«No se pudo conectar con el asistente…»**. *Solución:* el proxy no está encendido (terminal D).
- **«Se alcanzó el límite gratuito de consultas a la IA…»**. *Solución:* se agotó la cuota del día; pasa al modo de reglas (sección «Si se agota la cuota gratuita»).
- **«Tu sesión caducó…»**. *Solución:* el token dura 30 minutos; inicia sesión de nuevo.
- **«No entendí tu pregunta…»**. *Solución:* estás en modo de reglas, que solo reconoce unas pocas frases (ver `mock-server/README.md`); con la clave de Gemini el proxy entiende más variantes.

✅ Antes de continuar, confirma que...

- ☐ El Asistente responde con tus datos reales.
- ☐ La terminal D muestra una petición por cada pregunta.
- ☐ `flutter test` da 20 pasando y 2 fallando.

Paso 5 — Confirmación humana antes de registrar (8 min)

Por qué se hace

La IA puede equivocarse. Si propone una acción que **cambia datos** (registrar un gasto), la app no la ejecuta sola: muestra una confirmación y solo si aceptas llama al backend. La IA propone, la persona decide y el backend valida.

Instrucciones

5.1 — Descomentar. En `lib/screens/assistant_screen.dart`, descomentar el bloque `TODO(sesion-09) Paso 5` (1 línea, justo debajo del bloque del Paso 4).

5.2 — Aplicar. En la terminal B, presionar `R`.

5.3 — Pedir un registro. En el Asistente escribe `Registra un almuerzo de 6.50 en comida` y toca **Preguntar**. Aparece el diálogo «Confirmar acción».

5.4 — Cancelar. Toca **Cancelar** y revisa la pestaña **Gastos**: no se creó nada.

5.5 — Registrar. Repite la pregunta (gasta otra consulta gratuita) y toca **Registrar**: el Asistente confirma y el gasto aparece en **Gastos**.

5.6 — Ver la petición. En la terminal A aparece `POST /gastos/ -> 201`.

5.7 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Cancelar no crea nada; registrar crea el gasto. La terminal A muestra el `201`. `flutter test` da **22 pasando y 0 fallando**.

Errores comunes

- **El diálogo no aparece aunque la respuesta propone una acción**. *Solución:* falta el bloque del Paso 5, o escribiste una frase que la IA no entendió como un registro (usa «Registra un … de … en …»).
- **Falla con `r` no definido al descomentar**. *Solución:* descomenta primero el bloque del Paso 4 (el del Paso 5 usa lo que ese bloque calcula).
- **No aparece el gasto en la lista**. *Solución:* toca el ícono de recargar de la pestaña Gastos.

✅ Antes de continuar, confirma que...

- ☐ Cancelar no creó ningún gasto; registrar sí.
- ☐ Viste `POST /gastos/ -> 201` en la terminal A.
- ☐ `flutter test` da 22 pasando.

Paso 6 — Errores: límite de 500 y proxy apagado (7 min)

Por qué se hace

Un asistente robusto también falla bien. Dos casos distintos: el **backend** rechaza una acción que la IA propuso (límite de 500) y el **proxy** no responde. Cada uno debe mostrar un mensaje claro, sin congelar la app.

Instrucciones

6.1 — Provocar el límite. En el Asistente escribe `Registra un gasto de 600 en comida`, toca **Preguntar** y luego **Registrar** (gasta una consulta gratuita).

6.2 — Leer el mensaje. Aparece el texto del servidor: *«Este gasto supera el límite de 500.0 para la categoría 'comida'»*. La IA propuso, tú confirmaste, y el backend validó.

6.3 — Apagar el proxy. En la terminal D presiona `Ctrl + C`.

6.4 — Preguntar sin proxy. En el Asistente toca **Preguntar**: aparece *«No se pudo conectar con el asistente...»* y la app sigue respondiendo.

6.5 — Volver a encender el proxy. En la misma terminal D (la clave sigue definida):

```
node mock-server.js
```

6.6 — Distinguir de dónde viene cada mensaje. El del 6.2 vino del **backend**; el del 6.4, de la **app** (el proxy no respondió). Si algún día ves *«Se alcanzó el límite gratuito...»*, vino del **proxy**, porque Google rechazó la consulta.

Resultado esperado

Dos mensajes distintos y claros: el del límite (del backend) y el de conexión (del proxy apagado). Ninguno muestra trazas técnicas. Sabes distinguir de qué servidor viene cada error: backend, proxy o proveedor de IA.

Errores comunes

- **No aparece el límite**. *Solución:* usa un monto mayor que 500 (el límite es por categoría y por usuario). Si la IA no propone la acción, repite con la frase exacta `Registra un gasto de 600 en comida`.
- **Con el proxy apagado, la app parece quedarse cargando**. *Solución:* espera hasta 20 segundos: es el tiempo máximo antes del mensaje de «tardó demasiado».
- `EADDRINUSE` **al encender el proxy de nuevo**. *Solución:* quedó un proxy anterior; ciérralo con `Ctrl + C`.
- **El proxy dice** `modo REGLAS` **al volver a encenderlo**. *Solución:* abriste una terminal nueva y la clave no está definida; repite el 2.4 antes del 6.5.

✅ Antes de continuar, confirma que...

- ☐ Viste el mensaje del servidor al superar el límite.
- ☐ Con el proxy apagado, la app muestra un mensaje claro.
- ☐ Volviste a encender el proxy.

Paso 7 — Datos mínimos: qué viaja al proxy (4 min)

Por qué se hace

La teoría insiste en enviar solo lo necesario. Aquí lo compruebas con tus ojos: el proxy registra cuántos bytes recibe, y el tamaño no depende de cuántos gastos tengas, porque solo recibe una fila por categoría.

Instrucciones

7.1 — Fijar un punto de comparación. En el Asistente, pregunta `¿En qué categoría gasto más?` y mira en la terminal D la línea `Petición: N bytes · campos: consulta, resumen · categorías en el resumen: 4`.

7.2 — Crear más gastos. En la pestaña **Gastos**, crea 3 gastos pequeños más.

7.3 — Preguntar de nuevo. Repite la misma pregunta y compara el tamaño: casi no cambia, aunque tengas más gastos.

7.4 — Buscar la clave en la app. En la terminal C (en PowerShell: `Get-ChildItem -Recurse lib | Select-String -Pattern "api_key|AIza"`):

```
grep -rniE "api[_-]?key|AIza" lib
```

No debe imprimir nada: la app no conoce ninguna clave.

7.5 — Ver dónde se menciona en el proxy. (En PowerShell: `Select-String -Path mock-server/mock-server.js -Pattern GEMINI_API_KEY`):

```
grep -n "GEMINI_API_KEY" mock-server/mock-server.js
```

Solo aparece el **nombre** de la variable, leída del entorno; el valor de tu clave no está escrito en ningún archivo.

7.6 — Responder en una frase: ¿qué datos viajan al proxy y cuáles no, y por qué importa que Google pueda usar las consultas del plan gratuito para mejorar sus productos?

7.7 — Responder en una frase: si la clave estuviera escrita dentro de la app en vez de vivir en el proxy, ¿qué podría hacer alguien que descargue la app?

Resultado esperado

El tamaño de la petición es casi constante (unos pocos cientos de bytes). Puedes explicar

para qué sirve el proxy y qué pasaría sin él (la clave quedaría al alcance de cualquiera que descargue la app). Puedes decir que viajan la pregunta y el resumen por categoría, y que no viajan la lista de gastos, las fechas, el correo ni el token en el cuerpo (el token solo va en la cabecera).

Errores comunes

- **El tamaño cambia mucho**. *Solución:* solo debería cambiar por la longitud de tu pregunta y por los números del resumen; si crece con cada gasto, algo está enviando la lista completa.
- **El primer `grep` encuentra algo**. *Solución:* revisa si es una clave real o solo la palabra «key» en un comentario o nombre; una clave real debe borrarse, crear otra en AI Studio y desactivar la anterior.

✅ Antes de continuar, confirma que...

- ☐ Viste que el tamaño de la petición casi no cambia al tener más gastos.
- ☐ El `grep` sobre `lib` no encontró ninguna clave.
- ☐ Puedes enumerar qué datos viajan al proxy y cuáles no.
- ☐ Puedes explicar en una frase para qué sirve el proxy y por qué la clave no puede ir en la app.

Paso 8 — El proxy consulta el MCP del backend y prueba de extremo a extremo (7 min)

Por qué se hace

Hasta ahora el resumen de gastos lo calcula la app y se lo manda al proxy; si alguien alterara la app, podría mandar números falsos. Ahora el proxy se lo pide **directamente al backend**, con tu mismo token, a través del servidor **MCP** del módulo 2: ese servidor publica «herramientas» con nombre, y el proxy llama a una sola, `resumen_gastos`, que solo lee.

> **Esto es una demo, no un agente.** Aquí el programa decide qué herramienta llamar y siempre es la misma. En un agente real, el *modelo* elegiría entre varias herramientas. El camino correcto (el que explica la teoría) es: el agente vive en el servidor, el modelo solo ve herramientas de **solo lectura** de una lista permitida, hay un tope de vueltas y de uso, y toda acción que cambia datos vuelve a la app para que **la persona**

> **confirme**, como en el Paso 5. Por eso la app no habla MCP: no es un agente y no debe tener la clave.

Al final corres una **prueba de extremo a extremo**: una sola orden que recorre toda la cadena (backend → inicio de sesión → MCP → proxy) y marca con ✅ o ❌ cada eslabón. Cada ejecución gasta **una** consulta del plan gratuito (no gasta ninguna en modo de reglas).

Instrucciones

8.1 — Correr la prueba antes de cambiar nada. En la terminal C, desde la carpeta raíz de tu proyecto, con el correo y la contraseña con los que inicias sesión en la app:

```
node mock-server/probar-e2e.js tu-correo@ejemplo.com tu-contraseña
```

8.2 — Leer el resultado. Deben salir ✅ en los pasos 1, 2 y 3 y un ❌ en el 4: el proxy todavía usa el resumen que manda quien lo llama. Al final dice `1 paso(s) con problemas`. Es lo esperado.

8.3 — Descomentar. En `mock-server/mock-server.js`, descomentar el bloque `TODO(sesion-09) Paso 8` (6 líneas, justo debajo de `let origen = 'app';`).

8.4 — Reiniciar el proxy. En la terminal D presiona `Ctrl + C` y vuelve a encenderlo (la clave sigue definida en esa terminal):

```
node mock-server.js
```

8.5 — Repetir la prueba. En la terminal C, la misma orden del 8.1:

```
node mock-server/probar-e2e.js tu-correo@ejemplo.com tu-contraseña
```

8.6 — Usarlo desde la app. En el Asistente pregunta `¿En qué categoría gasto más?`. En la terminal D aparece `Resumen obtenido del backend por MCP (resumen_gastos)` y en la terminal A, líneas `POST /mcp/ -> 200`.

8.7 — Responder en una frase: ¿por qué esto todavía no es un agente y qué faltaría para que lo fuera de forma segura?

Resultado esperado

La prueba del 8.5 muestra cuatro ✅ y termina con `Todo bien: la cadena completa`

`funciona.` En la terminal D se ve el mensaje del MCP cada vez que preguntas. `flutter test` sigue dando 22 pasando (este paso no toca la app).

Errores comunes

- **El ❌ del paso 2 dice que no se pudo iniciar sesión**. *Solución:* revisa el correo y la contraseña: son los de tu usuario en la app, no los del docente.
- **El ❌ del paso 4 sigue saliendo después de descomentar**. *Solución:* no reiniciaste el proxy (8.4), o quedaron líneas sin descomentar: debe quedar sin `//` todo el bloque, hasta la llave de cierre `}`.
- **En la terminal D sale `MCP no disponible (...): se usa el resumen que envió la app`**. *Solución:* el proxy no pudo llegar al MCP (backend apagado o token caducado: 30 minutos). El asistente sigue funcionando con el resumen de la app. Revisa la terminal A e inicia sesión de nuevo en la app.
- **El ❌ del paso 4 dice que se agotó la cuota**. *Solución:* usa el modo de reglas (sección «Si se agota la cuota gratuita») y repite la prueba.
- **`EADDRINUSE` al encender el proxy**. *Solución:* quedó un proxy anterior abierto; ciérralo con `Ctrl + C`.

✅ Antes de continuar, confirma que...

- ☐ La prueba de extremo a extremo muestra cuatro ✅.
- ☐ Viste `Resumen obtenido del backend por MCP` en la terminal D.
- ☐ Puedes explicar en una frase la diferencia entre esta demo y un agente.

Paso 9 (opcional) — Preguntas sugeridas (8 min)

> Este paso **no cuenta dentro de los 60 minutos** ni bloquea la Sesión 10. Misma mecánica: descomentar y probar. Requiere el Paso 4 resuelto y el proxy encendido.

Por qué se hace

Una caja de texto vacía obliga a inventar qué preguntar: es el «problema de la página en blanco» de los asistentes de IA. Los chips con preguntas sugeridas escriben una consulta de ejemplo y la envían por el mismo camino que el botón: no hay un segundo camino hacia el proxy.

Instrucciones

9.1 — Descomentar. En `lib/screens/assistant_screen.dart`, descomentar el bloque `TODO(sesion-09) OPCIONAL Paso 9` (17 líneas, debajo del campo de texto).

9.2 — Aplicar. En la terminal B, presionar `R`.

9.3 — Usar un chip. Aparecen tres chips. Toca el primero: la pregunta se escribe y se envía sola.

9.4 — Probar el tercero. Toca «¿Qué lugar barato de comida me recomiendas?»: el proxy no conoce los lugares de la app, así que la respuesta será genérica o dirá que solo puede ayudar con los gastos. La interfaz debe tolerar respuestas imprecisas.

Resultado esperado

Cada chip llena el campo y dispara la consulta. Mientras llega la respuesta, los chips y el botón están deshabilitados.

Errores comunes

- **No aparecen los chips**. *Solución:* falta descomentar el bloque completo, hasta la última línea.
- **Los chips no hacen nada**. *Solución:* falta el Paso 4 o el proxy está apagado.

✅ Antes de continuar, confirma que...

- ☐ Al tocar un chip, la pregunta se escribe y se envía.
- ☐ Puedes explicar por qué los chips no llaman al proxy directamente.

Mini-reto *(sin solución escrita)*

Un cuarto chip. Agrega una pregunta propia (por ejemplo, «¿Cuánto llevo gastado en transporte?»). Pista: es solo otro texto en la lista; ¿qué tendría que cambiar en el proxy para que la reconozca?

Cierre — Guardar tu práctica: commit y push de `exploraec` (3 min)

Por qué se hace

Como en las sesiones anteriores, se guarda el trabajo en tu repositorio: queda respaldado y la

Sesión 10 parte de un estado conocido. Se guarda **solo la app**; el backend es otro repositorio.

Instrucciones

1. Ver qué cambió (no debe aparecer `proyecto-curso-spec-kit`):

```
git status --short
```

2. Guardar y subir:

```
git add -A
git commit -m "Sesión 9: asistente ExploraIA"
git push
```

3. Confirmar:

```
git status -sb
```

Resultado esperado

`git status -sb` muestra `## main...origin/main` **sin** la palabra `ahead`.

Errores comunes

- **`Author identity unknown`**. *Solución:* configurar `git config --global user.name` y `user.email`.
- **`no upstream branch`**. *Solución:* `git push -u origin main`.
- **`rejected`**. *Solución:* `git pull --rebase` y luego `git push`.

✅ Antes de terminar la sesión, confirma que...

- ☐ `git status -sb` muestra `## main...origin/main` sin `ahead`.
- ☐ Detuviste el proxy (`Ctrl + C` en la terminal D) y el backend si ya no los necesitas.
- ☐ Tu clave no está en ningún archivo (`git status --short` no muestra nada raro). Cuando termine el curso, puedes borrarla desde AI Studio.

Entregable de la práctica

- Una clave gratuita de Google creada por ti y guardada solo en una variable de entorno del proxy.
- Pestaña «Asistente ExploraIA» funcional, conectada al proxy local y al backend de gastos.
- Respuestas que coinciden con tus datos reales.
- Confirmación explícita antes de registrar un gasto propuesto por la IA.
- Mensajes claros para el límite de 500 y para el proxy apagado.
- Ninguna clave de IA en el código ni en el repositorio; solo el resumen por categoría viaja al proxy.
- Un proxy que pide tu resumen al servidor MCP del backend (demo) y una prueba de extremo a extremo con cuatro ✅.
- *(Opcional)* Preguntas sugeridas (Paso 9).

Conexión con la siguiente sesión

La Sesión 10 no agrega tecnología nueva: cierra el curso con calidad y entrega. Verás cómo capturar errores inesperados, probar tu código, auditar secretos, generar un APK con **GitHub Actions** y preparar la entrega del Proyecto Final en Moodle. Llegar con el Asistente funcionando (con la clave de Gemini o en modo de reglas) es el punto de partida.

Anexo — Todos los comandos de la práctica, en orden

Resumen de consulta (Linux/Mac). Las ediciones de archivo son siempre «descomentar el bloque `TODO(sesion-09) Paso N`»: no son comandos. `<id>` es el que muestra `flutter emulators`.

Paso 0: abrir https://aistudio.google.com/apikey, iniciar sesión, pulsar **Create API key** y copiarla.

Paso 1

```
git status
git add -A
git commit -m "Sesión 8"
docker compose up                       # terminal A, en proyecto-curso-spec-kit
curl http://localhost:8000/health       # terminal C
flutter emulators --launch <id>
flutter run                             # terminal B
node --version
```

Punto de partida y Pasos 2 a 3

```
git fetch starter
git checkout starter/sesion-09 -- lib test pubspec.yaml PLACEHOLDERS.md mock-server
flutter pub get
flutter test                            # 19 pasan, 3 fallan
cd mock-server                          # terminal D
export GEMINI_API_KEY="pega-aquí-tu-clave"
node mock-server.js
curl -X POST -H "Content-Type: application/json" -d @mock-server/ejemplo-registro.json
 http://localhost:3000/asistente
```

Pasos 4 a 7 (después de descomentar: `R` en la terminal B)

```
flutter test                            # 20 pasan al terminar el Paso 4; 22 al
terminar el Paso 5
node mock-server.js                     # Paso 6, para volver a encender el proxy
grep -rniE "api[_-]?key|AIza" lib
grep -n "GEMINI_API_KEY" mock-server/mock-server.js
```

Paso 8 (después de descomentar el bloque en `mock-server/mock-server.js`, reinicia el proxy con `Ctrl + C`)

```
node mock-server/probar-e2e.js tu-correo@ejemplo.com tu-contraseña   # terminal C,
antes: 3 ✅ y 1 ❌; después: 4 ✅
node mock-server.js                                                  # terminal D,
para reiniciar el proxy
```

Si se agota la cuota (terminal D)

```
unset GEMINI_API_KEY
node mock-server.js
```

Cierre

```
git status --short
git add -A
git commit -m "Sesión 9: asistente ExploraIA"
git push
git status -sb
```